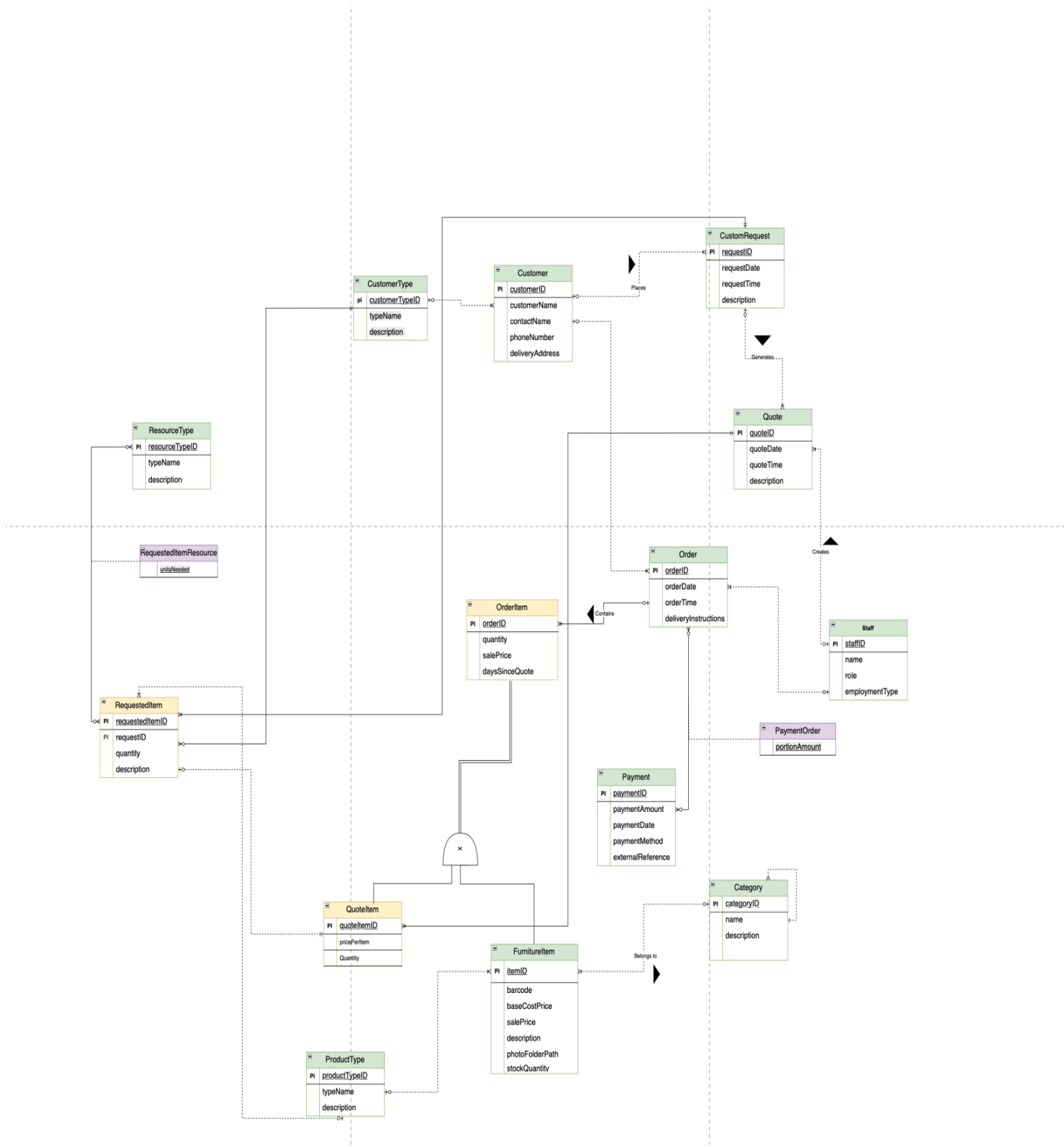


Task A – Conceptual Modelling

This task presents the conceptual data model for the Wonky Furniture case study. The model has been developed using Crow's Foot notation to represent entities, relationships, and constraints. It includes strong entities, weak entities, and associative entities as required by the problem domain. Optional and mandatory participation have been indicated for each relationship, and identifying relationships are distinguished from non-identifying ones.

Following the EER diagram, a numbered list of assumptions has been provided. These assumptions document the decisions made to resolve gaps and ambiguities in the case study and explain the impact each assumption has on the final model.

EER Diagram



Assumptions for task A:

1. Every furniture item must belong to exactly one category.

A category may exist without items, but no furniture item can exist without being categorized.

Impact: Added a mandatory relationship from FurnitureItem to Category.

2. Each category may or may not have a parent category.

This allows top-level categories (root categories) to exist without a parent.

Impact: Modelled as a recursive relationship on Category with optional parent participation.

3. Each furniture item must have one product type.

A product type may exist without associated items (e.g., a new type yet to be stocked).

Impact: Added a mandatory relationship from FurnitureItem to ProductType.

4. Every customer must belong to exactly one customer type.

A customer type may exist without customers.

Impact: Mandatory relationship from Customer to CustomerType.

5. A customer may exist without ever placing an order or making a custom request.

This reflects potential leads or new customers who have not yet purchased.

Impact: Optional relationships from Customer to Order and Customer to CustomRequest.

6. Every custom request must contain at least one requested item.

A requested item cannot exist without a parent request.

Impact: Weak entity RequestedItem with total participation in CustomRequest.

7. Every requested item must use at least one resource type.

A resource type may exist without being used yet.

Impact: Many-to-many relationship between RequestedItem and ResourceType, resolved via associative entity RequestedItemResource.

8. Every quote must be created by exactly one staff member and linked to exactly one custom request.

A custom request may or may not have a quote.

Impact: Added mandatory relationship from Quote to StaffMember, and optional relationship from CustomRequest to Quote.

9. Every quote must contain at least one quote item.

A quote item cannot exist without a parent quote.

Impact: Weak entity QuoteItem with total participation in Quote.

10. Every order must be created by exactly one staff member and linked to exactly one customer.

A staff member or customer may exist without orders.

Impact: Added mandatory relationship from Order to both Customer and StaffMember.

11. Every order must contain at least one order item.

An order item cannot exist without its parent order.

Impact: Weak entity OrderItem with total participation in Order.

12. A payment must always be linked to at least one order.

An order may exist with no payments yet.

Impact: Many-to-many relationship between Payment and Order, resolved via associative entity PaymentOrder.

13. Each order item must be either a furniture item or a quoted item, but not both.

Impact: Modeled as a disjoint, total specialization of OrderItem into FurnitureOrderItem and QuotedOrderItem.

Task B & C

Tasks B and C focus on implementing and testing the database. In Task B, the physical model provided in the brief was translated into SQL **CREATE TABLE** statements with the required primary keys, foreign keys, and constraints, followed by the insertion of test data to populate the database. Screenshots of the MySQL results area are included to show successful execution. Task C then builds on this populated database by running SQL queries to answer the case study questions. Each query is shown along with its result set, demonstrating the ability to extract and analyse information from the database.

Screenshots for tasks B & C:

screenshot of the MySQL results area showing tables created and data inserted.

Output			
Action Output			
#	Time	Action	Message
✓ 1	09:49:27	use '48713406'	0 row(s) affected
✓ 2	09:49:36	use '48713406'	0 row(s) affected
✓ 3	09:49:36	CREATE TABLE CustomerType (customerTypeID INT PRIMARY KEY, customerTypeName VARCHAR(45) NOT NULL, customerTypeDescription...	0 row(s) affected
✓ 4	09:49:36	CREATE TABLE Customer (customerID INT PRIMARY KEY, customerName VARCHAR(300) NOT NULL, contactName VARCHAR(300), custo...	0 row(s) affected
✓ 5	09:49:36	CREATE TABLE StaffMember (staffID INT PRIMARY KEY, staffName VARCHAR(300) NOT NULL, staffRole VARCHAR(30), staffCurrentEmplo...	0 row(s) affected
✓ 6	09:49:36	CREATE TABLE `Order` (orderID INT PRIMARY KEY, orderDate DATE NOT NULL, orderTime TIME NOT NULL, orderDeliveryInstructions TE...	0 row(s) affected
✓ 7	09:49:36	CREATE TABLE OrderItem (orderID INT NOT NULL, itemSequenceNumber INT NOT NULL, quantity INT NOT NULL, salePricePerItem DECI...	0 row(s) affected
✓ 8	09:49:36	CREATE TABLE Payment (paymentID INT PRIMARY KEY, paymentAmount DECIMAL(10,2) NOT NULL, paymentDateTime DATETIME NOT NU...	0 row(s) affected
✓ 9	09:49:36	CREATE TABLE PaymentPortion (paymentID INT NOT NULL, orderID INT NOT NULL, paymentPortionAmount DECIMAL(10,2) NOT NULL, P...	0 row(s) affected
✓ 10	09:49:36	INSERT INTO CustomerType VALUES (1, 'Individual', 'Single person ordering'), (2, 'Company', 'Business account')	2 row(s) affected Records: 2 Duplicates: 0 Warnings: 0
✓ 11	09:49:36	INSERT INTO Customer VALUES (1, 'Alice Smith', 'Alice Smith', '0412345678', '123 Main St', 1), (2, 'Zara Interiors', 'Zara Lee', '0498765432', '456 High St', ...	3 row(s) affected Records: 3 Duplicates: 0 Warnings: 0
✓ 12	09:49:36	INSERT INTO StaffMember VALUES (1, 'Jane Doe', 'Sales Rep', 'Full-time'), (2, 'Tom Lee', 'Manager', 'Part-time')	2 row(s) affected Records: 2 Duplicates: 0 Warnings: 0
✓ 13	09:49:36	INSERT INTO `Order` VALUES (1, '2023-03-10', '09:30:00', 'Leave at door', 1, 1), (2, '2024-05-15', '14:00:00', 'Deliver upstairs', 2, 1), (3, '2025-02-20', '10:1...	3 row(s) affected Records: 3 Duplicates: 0 Warnings: 0
✓ 14	09:49:36	INSERT INTO OrderItem VALUES (1, 1, 2, 150.00), (1, 2, 1, 200.00), (2, 1, 3, 75.00), (3, 1, 1, 500.00)	4 row(s) affected Records: 4 Duplicates: 0 Warnings: 0
✓ 15	09:49:36	INSERT INTO Payment VALUES (1, 500.00, '2023-03-12 11:00:00', 'CreditCard', 'REF123'), (2, 100.00, '2024-05-16 15:30:00', 'PayPal', 'PP987'), (3, 250.0...	3 row(s) affected Records: 3 Duplicates: 0 Warnings: 0
✓ 16	09:49:36	INSERT INTO PaymentPortion VALUES (1, 1, 350.00), (1, 2, 150.00), (2, 2, 100.00), (3, 3, 250.00)	4 row(s) affected Records: 4 Duplicates: 0 Warnings: 0

Output		
History Output		
Date	Time	SQL
2025-09-09	10:00:06	SELECT o.orderID, SUM(o.quantity * oi.salePricePerItem) AS totalOrderAmount, COALESCE(SUM(pp.paymentPortionAmount), 0) AS totalPaid, SUM(o.quantity * oi.salePricePerItem) - COALESCE(SUM(pp.paymentPortionAmount), 0) AS totalOwing FROM `Order` o JOIN OrderItem oi ON o.orderID = oi.orderID GROUP BY o.orderID
	09:58:42	SELECT o.orderID, COALESCE(SUM(pp.paymentPortionAmount), 0) AS totalPaid FROM `Order` o LEFT JOIN PaymentPortion pp ON o.orderID = pp.orderID GROUP BY o.orderID
	09:57:10	SELECT o.orderID, o.orderDate, o.orderTime, c.customerName, c.contactName, SUM(o.quantity * oi.salePricePerItem) AS totalAmount FROM `Order` o JOIN Customer c ON o.customerID = c.customerID JOIN OrderItem oi ON o.orderID = oi.orderID GROUP BY o.orderID, c.customerName
	09:56:16	SELECT c.customerID, c.customerName, MIN(o.orderDate) AS firstOrderDate FROM Customer c LEFT JOIN `Order` o ON c.customerID = o.customerID GROUP BY c.customerID, c.customerName
	09:55:18	SELECT customerID, customerName, customerPhoneNumber, customerAddress FROM Customer WHERE customerName LIKE "%a%" OR customerName LIKE "%z%" ORDER BY customerName DESC
	09:53:15	SELECT o.orderID, SUM(o.quantity * oi.salePricePerItem) AS totalOrderAmount, COALESCE(SUM(pp.paymentPortionAmount), 0) AS totalPaid, SUM(o.quantity * oi.salePricePerItem) - COALESCE(SUM(pp.paymentPortionAmount), 0) AS totalOwing FROM `Order` o JOIN OrderItem oi ON o.orderID = oi.orderID GROUP BY o.orderID
	09:53:09	CREATE TABLE CustomerType (customerTypeID INT PRIMARY KEY, customerTypeName VARCHAR(45) NOT NULL, customerTypeDescription TEXT)
	09:53:09	use '48713406'
	09:49:36	INSERT INTO PaymentPortion VALUES (1, 1, 350.00), (1, 2, 150.00), (2, 2, 100.00), (3, 3, 250.00)
	09:49:36	INSERT INTO Payment VALUES (1, 500.00, '2023-03-12 11:00:00', 'CreditCard', 'REF123'), (2, 100.00, '2024-05-16 15:30:00', 'PayPal', 'PP987'), (3, 250.00, '2025-02-21 12:00:00', 'Bank Transfer', 'BT456)
	09:49:36	INSERT INTO OrderItem VALUES (1, 1, 2, 150.00), (1, 2, 1, 200.00), (2, 1, 3, 75.00), (3, 1, 1, 500.00)
	09:49:36	INSERT INTO `Order` VALUES (1, '2023-03-10', '09:30:00', 'Leave at door', 1, 1), (2, '2024-05-15', '14:00:00', 'Deliver upstairs', 2, 1), (3, '2025-02-20', '10:15:00', 'Handle with care', 3, 2)
	09:49:36	INSERT INTO StaffMember VALUES (1, 'Jane Doe', 'Sales Rep', 'Full-time'), (2, 'Tom Lee', 'Manager', 'Part-time')
	09:49:36	INSERT INTO Customer VALUES (1, 'Alice Smith', 'Alice Smith', '0412345678', '123 Main St', 1), (2, 'Zara Interiors', 'Zara Lee', '0498765432', '456 High St', 2), (3, 'Bob Johnson', 'Bob Johnson', '0411222333', '789 Park Rd', 1)
	09:49:36	INSERT INTO CustomerType VALUES (1, 'Individual', 'Single person ordering'), (2, 'Company', 'Business account')
	09:49:36	CREATE TABLE PaymentPortion (paymentID INT NOT NULL, orderID INT NOT NULL, paymentPortionAmount DECIMAL(10,2) NOT NULL, PRIMARY KEY (paymentID, orderID), FOREIGN KEY (paymentID) REFERENCES Payment(paymentID), FOREIGN KEY (orderID) REFERENCES `Order`(orderID))
	09:49:36	CREATE TABLE Payment (paymentID INT PRIMARY KEY, paymentAmount DECIMAL(10,2) NOT NULL, paymentDateTime DATETIME NOT NULL, paymentMethod VARCHAR(20), paymentSystemReferenceNumber VARCHAR(256))
	09:49:36	CREATE TABLE OrderItem (orderID INT NOT NULL, itemSequenceNumber INT NOT NULL, quantity INT NOT NULL, salePricePerItem DECIMAL(10,2) NOT NULL, PRIMARY KEY (orderID, itemSequenceNumber), FOREIGN KEY (orderID) REFERENCES `Order`(orderID))
	09:49:36	CREATE TABLE `Order` (orderID INT PRIMARY KEY, orderDate DATE NOT NULL, orderTime TIME NOT NULL, orderDeliveryInstructions TEXT, customerID INT NOT NULL, createdByStaffID INT NOT NULL, FOREIGN KEY (customerID) REFERENCES Customer(customerID), FOREIGN KEY (staffID) REFERENCES StaffMember(staffID))
	09:49:36	CREATE TABLE StaffMember (staffID INT PRIMARY KEY, staffName VARCHAR(300) NOT NULL, staffRole VARCHAR(30), staffCurrentEmployeeType VARCHAR(20))
	09:49:36	CREATE TABLE Customer (customerID INT PRIMARY KEY, customerName VARCHAR(300) NOT NULL, contactName VARCHAR(300), customerPhoneNumber VARCHAR(20), customerAddress VARCHAR(500), customerTypeID INT, FOREIGN KEY (customerTypeID) REFERENCES CustomerType(customerTypeID))
	09:49:36	CREATE TABLE CustomerType (customerTypeID INT PRIMARY KEY, customerTypeName VARCHAR(45) NOT NULL, customerTypeDescription TEXT)
	09:49:36	use '48713406'
	09:49:27	use '48713406'

A screenshot of the MySQL results area and returned data for each query.

Task(i)

117

```
120 • SELECT customerID, customerName, customerPhoneNumber, customerAddress
121 FROM Customer
122 WHERE customerName LIKE '%a%' OR customerName LIKE '%z%'
123 ORDER BY customerName DESC;
```

Result Grid				
		Filter Rows:	Edit:	
		Export/Import:		Wrap Cell
	customerID	customerName	customerPhoneNumber	customerAddress
▶	2	Zara Interiors	0498765432	456 High St
	1	Alice Smith	0412345678	123 Main St
*	NULL	NULL	NULL	NULL

Task(ii)

```
126 -- Task C (ii)
127
128 • SELECT c.customerID, c.customerName, MIN(o.orderDate) AS firstOrderDate
129 FROM Customer c
130 LEFT JOIN `Order` o ON c.customerID = o.customerID
131 GROUP BY c.customerID, c.customerName;
```

Result Grid			
		Filter Rows:	Export:
		Wrap Cell Content:	
	customerID	customerName	firstOrderDate
▶	1	Alice Smith	2023-03-10
	2	Zara Interiors	2024-05-15
	3	Bob Johnson	2025-02-20

Task(iii)

```

136  -- Task C (iii)
137  •  SELECT o.orderID,
138         o.orderDate,
139         o.orderTime,
140         c.customerName,
141         c.contactName,
142         SUM(oi.quantity * oi.salePricePerItem) AS totalAmount
143  FROM `Order` o
144  JOIN Customer c ON o.customerID = c.customerID
145  JOIN OrderItem oi ON o.orderID = oi.orderID
146  GROUP BY o.orderID, o.orderDate, o.orderTime, c.customerName, c.contactName
147  ORDER BY o.orderDate ASC, c.customerID DESC;

```

Result Grid

Filter Rows:

Export:

Wrap Cell Content:

	orderID	orderDate	orderTime	customerName	contactName	totalAmount
▶	1	2023-03-10	09:30:00	Alice Smith	Alice Smith	500.00
	2	2024-05-15	14:00:00	Zara Interiors	Zara Lee	225.00
	3	2025-02-20	10:15:00	Bob Johnson	Bob Johnson	500.00

Task(iv)

```

148
149  -- Task C (iv)
150  •  SELECT o.orderID,
151         COALESCE(SUM(pp.paymentPortionAmount), 0) AS totalPaid
152  FROM `Order` o
153  LEFT JOIN PaymentPortion pp ON o.orderID = pp.orderID
154  GROUP BY o.orderID;

```

Result Grid

Filter Rows:

Export:

Wrap Cell Content:

	orderID	totalPaid
▶	1	350.00
	2	250.00
	3	250.00

Task(v)

```

157      -- Task C (v)
158 •   SELECT s.staffID,
159          s.staffName,
160          y.year,
161          COALESCE(COUNT(o.orderID), 0) AS numberOfOrders
162   FROM StaffMember s
163   CROSS JOIN (
164       SELECT 2021 AS year UNION
165       SELECT 2022 UNION
166       SELECT 2023 UNION
167       SELECT 2024 UNION
168       SELECT 2025
169   ) y
170   LEFT JOIN `Order` o
171       ON o.createdByStaffID = s.staffID
172      AND YEAR(o.orderDate) = y.year
173   GROUP BY s.staffID, s.staffName, y.year
174   ORDER BY s.staffID, y.year;
175

```

Result Grid			Filter Rows:	Export:	Wrap Cell Content:
	orderID	totalPaid			
▶	1	350.00			
	2	250.00			
	3	250.00			

Task(vi)

```

175
176 -- Task C (vi)
177 • SELECT o.orderID,
178         SUM(oi.quantity * oi.salePricePerItem) AS totalOrderAmount,
179         COALESCE(SUM(pp.paymentPortionAmount), 0) AS totalPaid,
180         SUM(oi.quantity * oi.salePricePerItem) - COALESCE(SUM(pp.paymentPortionAmount), 0) AS totalOwing
181 FROM `Order` o
182 JOIN OrderItem oi ON o.orderID = oi.orderID
183 LEFT JOIN PaymentPortion pp ON o.orderID = pp.orderID
184 GROUP BY o.orderID;
185
186
187
188

```

Result Grid |  Filter Rows: | Export:  | Wrap Cell Content: 

	orderID	totalOrderAmount	totalPaid	totalOwing
▶	1	500.00	700.00	-200.00
	2	450.00	250.00	200.00
	3	500.00	250.00	250.00